

Averon v4.0.0

Enterprise Blockchain Asset Tokenization Platform with AI Analysis. A comprehensive technical reference covering blockchain layer, service architecture, database schema, security model, and deployment infrastructure.

Real-World Asset (RWA) Tokenization
Custom Proof-of-Work Blockchain
Google Gemini AI Integration

Table of Contents

1. High-Level Overview	3
2. System Architecture	3
2.1 Architecture Diagram	3
2.2 Technology Stack	4
2.3 Project Structure	5
3. Blockchain Layer	5
3.1 Transaction Types	5
3.2 Block Mining	6
3.3 Wallet System	6
3.4 Merkle Tree and Consensus	7
4. Database Layer	7
4.1 Table Inventory	7
5. Service Layer - Major Flows	8
5.1 User Registration and Authentication	8
5.2 Asset Tokenization Lifecycle	9
5.2.1 Creation Through AI Analysis	9
5.2.2 Tokenization, Investment, and Payout	10
5.3 Payment and Coin Minting	10
5.4 Trading Engine	10
5.5 KYC and AML Compliance	10
5.6 Withdrawal and Settlement	10
5.7 Admin Operations	11
6. Middleware Stack	11
6.2 Audit Log and Input Validation	11
7. Real-Time Layer	11
8. Price Engine	12

9. Fee Structure	12
10. Security Model	13
11. Deployment Architecture	13
12. API Route Map	14
13. Initialization Sequence	15

1. High-Level Overview

Averon is a full-stack **real-world asset (RWA) tokenization platform** that bridges traditional physical assets with blockchain technology. The platform enables users to register accounts with blockchain wallets, purchase native platform coins (Averon Coins, or AC) using fiat currency through multiple payment gateways, create and tokenize real-world assets such as land, vehicles, and stocks, and trade asset tokens on an integrated exchange with circuit breaker protection. The system is built on a custom Proof-of-Work blockchain with ECDSA secp256k1 cryptography, providing enterprise-grade security and full on-chain transparency for all transactions.

At its core, Averon provides nine primary capabilities. First, users can **register** and automatically receive a blockchain wallet using ECDSA secp256k1 key pairs. Second, they can **purchase Averon Coins (AC)** using fiat currencies such as INR, USD, EUR, and GBP through Razorpay, Stripe, UPI, or Wire transfer. Third, users can **create real-world assets** by providing details like title, description, category, and raise amount, along with supporting documentation. Fourth, the platform performs **AI-powered analysis** of uploaded assets using Google Gemini 2.0 Flash (with a fallback engine) for verification, valuation, and risk scoring. Fifth, verified assets are **tokenized** into fractional tokens recorded on the blockchain.

Sixth, users can **buy and sell asset tokens** using AC through an escrow-backed funding mechanism that ensures atomic, race-condition-free transactions. Seventh, the platform offers an integrated **order-book exchange** for trading AC with price-time priority matching and a circuit breaker that halts trading if price moves more than 10% within one hour. Eighth, users can **withdraw** by converting AC back to fiat with full KYC/AML compliance checks enforced at every step. Ninth, a comprehensive **admin panel** provides full platform oversight including user management, compliance review, withdrawal processing, and real-time dashboard statistics.

2. System Architecture

The Averon platform follows a layered architecture pattern with five distinct layers: Client, Server, Service, Blockchain, and Data. Each layer encapsulates a specific set of responsibilities and communicates with adjacent layers through well-defined interfaces. A separate Real-Time layer provides event-driven communication across the entire stack. This separation of concerns ensures modularity, testability, and the ability to scale individual components independently as the platform grows.

2.1 Architecture Diagram

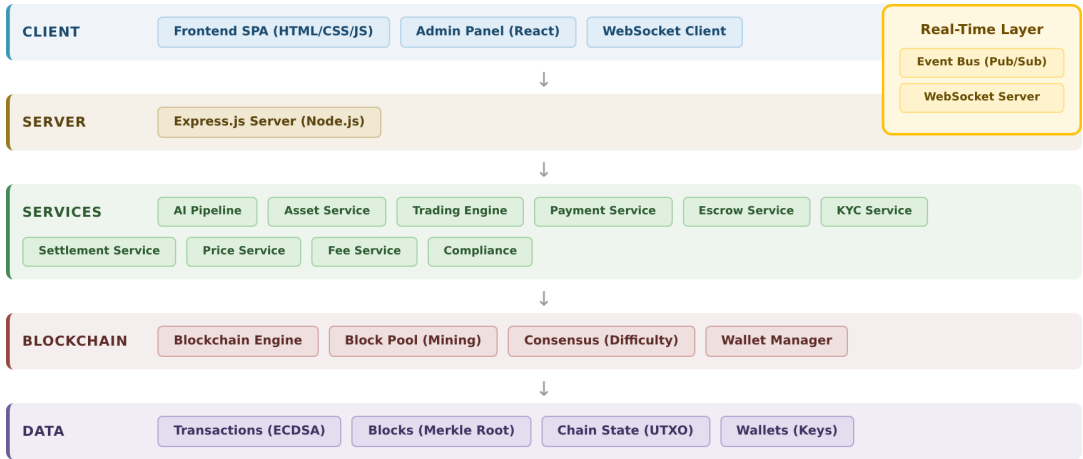


Figure 1. Averon v4.0.0 layered system architecture showing all five primary layers and the real-time event bus.

2.2 Technology Stack

The technology stack has been carefully selected to minimize external dependencies while maximizing performance and security. The entire backend runs on Node.js 18+ with ES2022+ features, using Express.js 4.x for the REST API server. The database layer uses sql.js (SQLite compiled to WebAssembly), providing zero native dependencies and in-process execution. The blockchain is a custom Proof-of-Work implementation using SHA-256 mining with Merkle trees and ECDSA signing, built entirely from scratch without external blockchain libraries.

Layer	Technology	Details
Runtime	Node.js >= 18	ES2022+, native fetch, crypto
Framework	Express.js 4.x	REST API server with middleware pipeline
Database	sql.js (SQLite WASM)	Zero native deps, in-process, auto-persist
Blockchain	Custom PoW Chain	SHA-256 mining, Merkle trees, ECDSA secp256k1
Cryptography	Node crypto	secp256k1 keys, PBKDF2, HMAC-SHA256
AI	Google Gemini 2.0 Flash	Multi-modal asset analysis with fallback
Payments	Razorpay + Stripe SDK	INR/USD/EUR/GBP payment processing
WebSocket	ws package	Real-time price/trade/block updates
File Upload	Multer	Document storage for asset verification
Auth	Custom JWT (zero-dep)	HMAC-SHA256 signed, PBKDF2 passwords

Layer	Technology	Details
Process Mgmt	PM2 / Node Cluster	Multi-worker production mode
Container	Docker + Compose	1GB memory limit, health checks
Frontend	Vanilla HTML/CSS/JS	SPA with WebSocket integration

Table 1. Complete technology stack used across the Averon platform.

2.3 Project Structure

The project is organized into a clear directory structure with separation between backend logic, frontend assets, data storage, scripts, and tests. The backend directory contains configuration files, middleware for authentication and rate limiting, the blockchain engine with all cryptographic primitives, and a comprehensive service layer. The constants file alone contains 264 lines of platform configuration, while the database initialization file defines over 25 table schemas across 701 lines of SQL.

Path	Description
server.js	Entry point - Express app, routes, initialization
backend/config/	constants.js (264 lines), database.js (701 lines, 25+ tables), security.js
backend/middleware/	auth.js, adminAuth.js, audit.js, rateLimiter.js, validator.js
backend/blockchain/	chain.js, block.js, transaction.js, merkle.js, consensus.js, wallet.js
backend/services/	11 service modules (AI, asset, trading, payment, escrow, KYC, etc.)
frontend/	index.html (16KB), style.css (22KB), app.js (45KB), ws.js, admin panel
data/	averon.db (SQLite), chain.json (blocks), wallets.json (ECDSA keys)
scripts/	seed.js, reset.js, benchmark.js, generate-history.js
tests/	blockchain.test.js, trading.test.js, api.test.js

Table 2. Key directories and files in the Averon project structure.

3. Blockchain Layer

The blockchain layer is the foundational infrastructure of the Averon platform, providing an immutable, transparent ledger for all financial transactions. Unlike platforms that rely on external blockchain networks, Averon implements a complete custom blockchain from scratch, giving full control over consensus parameters, transaction types, and mining behavior. The blockchain uses a UTXO (Unspent Transaction Output) model similar to Bitcoin, where balances are computed by scanning all blocks for unspent outputs rather than maintaining explicit balance fields.

3.1 Transaction Types

The blockchain supports twelve distinct transaction types, each serving a specific purpose within the platform ecosystem. The MINT transaction type handles coin creation from fiat purchases, while TRANSFER covers standard user-to-user or user-to-system movements. The INVEST and DIVEST types manage the flow of funds into and out of asset escrow accounts. PAYOUT transactions release escrow funds to asset owners when assets are fully funded, and REFUND transactions return funds to investors when assets expire without reaching their funding target.

Type	Description
MINT	Coin creation from fiat purchase
TRANSFER	User-to-user or user-to-system movement
INVEST	User to Escrow (buying asset tokens)
DIVEST	Exit from investment
PAYOUT	Escrow to Owner (asset fully funded)
REFUND	Escrow to Investors (asset expired)
FEE	Fee collection (trading, listing, raise)
ASSET_CREATE	Asset tokenization recorded on chain
ASSET_VERIFY	AI verification result recorded
ASSET_CLOSE	Asset lifecycle completed
TRADE	AC exchange between traders
REWARD	Mining reward (0.1 AC per block)

Table 3. All twelve blockchain transaction types.

3.2 Block Mining

Blocks are mined using a SHA-256 Proof-of-Work algorithm with dynamic difficulty adjustment. The target block time is 30 seconds, with difficulty adjusting every 10 blocks based on actual mining speed. If blocks are mined too fast (less than half the expected time), difficulty increases; if too slow (more than double), difficulty decreases. The difficulty range spans 1 to 6 leading zeros, with a maximum of 100 transactions per block and a 1 MB block size limit. Each successfully mined block rewards the system wallet with 0.1 AC.

3.3 Wallet System

The wallet system uses the secp256k1 elliptic curve, the same cryptographic curve employed by Bitcoin and Ethereum. Key pairs are generated and stored in PEM format (SPKI for public keys, PKCS8 for private keys). Wallet addresses are derived by computing SHA-256 of the public key, followed by RIPEMD-160, and prepending a "0x" prefix. The platform maintains three categories of special wallets: the __SYSTEM__ wallet handles coin minting and mining rewards, the __PLATFORM_FEE__ wallet

collects all platform fees, and per-asset escrow wallets (ESCROW_assetId_timestamp) hold investor funds during funding periods.

3.4 Merkle Tree and Consensus

Each block contains a Merkle root computed from all transaction hashes using a binary tree structure with SHA-256 hashing. Odd-leaf counts trigger duplication of the last leaf. The tree supports proof generation (getProof) and verification (verify), enabling efficient single-transaction verification without downloading the full block. Chain validation enforces six critical checks: index continuity, hash chain linkage, hash integrity, difficulty validation, Merkle root verification, and timestamp ordering. Fork resolution follows the longest valid chain principle. The balance model uses UTXO-style scanning across all blocks combined with deductions for pending outgoing transactions.

4. Database Layer

The database layer uses sql.js (SQLite compiled to WebAssembly), providing a fully functional relational database that runs entirely in-process without native C dependencies. The database auto-saves to data/averon.db every 3 seconds and supports full transaction semantics with BEGIN/COMMIT/ROLLBACK. The schema comprises 28 tables covering user accounts, blockchain wallets, asset management, escrow operations, trading, payments, KYC compliance, and system administration.

4.1 Table Inventory

#	Table	Purpose	Key Columns
1	users	User accounts	id, email, password_hash, role, wallet_address, kyc_status
2	sessions	JWT refresh sessions	user_id, refresh_token, expires_at, is_revoked
3	wallets	ECDSA key pairs	user_id, public_key, private_key, address
4	assets	Tokenized assets	owner_id, title, category, status, ai_*, token_count
5	asset_documents	Uploaded docs	asset_id, filepath, mimetype, doc_hash
6	asset_status_history	State transitions	asset_id, old_status, new_status, changed_by
7	asset_valuations	AI valuations	asset_id, valuation, risk_score, confidence
8	asset_tokens	Fractional tokens	asset_id, token_index, price, owner_id, tx_hash
9	escrow_accounts	Per-asset escrow	asset_id, address, balance, total_received
10	escrow_transactions	Escrow movements	type (LOCK/RELEASE/REFUND), amount, tx_hash

#	Table	Purpose	Key Columns
11	coin_orders	Trading order book	side, type, amount, price, filled, status
12	coin_trades	Executed trades	buy_order_id, sell_order_id, amount, price, fees
13	economy	Global metrics	price, total_supply, circulating_supply, market_cap
14	price_history	Price time series	price, volume, high, low, open, close
15	fee_ledger	All collected fees	user_id, fee_type, amount, reference_id
16	notifications	User notifications	user_id, type, title, message, is_read
17	payment_gateways	Gateway configs	name, provider, currencies, min/max, fees
18	payment_orders	Fiat to AC purchases	user_id, gateway, fiat_amount, coin_amount, status
19	payment_transactions	Payment audit trail	order_id, type, gateway_tx_id, status
20	kyc_records	KYC documents	user_id, doc_type, doc_number, doc_status, tier
21	kyc_tier_history	Tier changes	user_id, old_tier, new_tier, reason
22	withdrawal_requests	AC to fiat	user_id, coin_amount, fiat_amount, bank_account
23	settlement_batches	Batch settlements	gateway, total_amount, status
24	reconciliation_log	Payment reconciliation	order_id, internal_amount, gateway_amount
25	daily_limits	KYC limit tracking	user_id, date, total_bought, tx_count
26	audit_log	Tamper-proof audit	action, details, prev_hash, entry_hash
27	system_config	Dynamic config	key, value, updated_by
28	activity_log	User activity feed	user_id, action, details, tx_hash, amount

Table 4. Complete database schema inventory with 28 tables.

5. Service Layer - Major Flows

The service layer implements all business logic through eleven dedicated service modules, each responsible for a specific domain. These services orchestrate complex multi-step workflows spanning database operations, blockchain transactions, external API calls, and real-time event broadcasting.

5.1 User Registration and Authentication

User registration is a multi-step process that simultaneously creates a user account, generates an ECDSA blockchain wallet, and establishes an authenticated session. The auth middleware hashes passwords using PBKDF2-SHA512 with 100,000 iterations, a 64-byte derived key, and a 32-byte random salt. The wallet

manager generates a secp256k1 key pair, derives the blockchain address using SHA-256 and RIPEMD-160 hashing, and stores both keys in the wallets table. The first registered user automatically receives the admin role. Authentication uses zero-dependency JWT tokens with HMAC-SHA256 signing. Access tokens expire in 15 minutes while refresh tokens last 7 days. Account lockout occurs after 5 failed login attempts with a 15-minute lockout period.

5.2 Asset Tokenization Lifecycle

The asset tokenization lifecycle is the heart of Averon, implemented as a 14-state finite state machine governing every asset from creation through completion or expiration. Each state transition is recorded in the asset_status_history table with the previous state, new state, actor, and reason. The state machine enforces strict rules about valid transitions, preventing illegal changes that could compromise tokenization integrity.

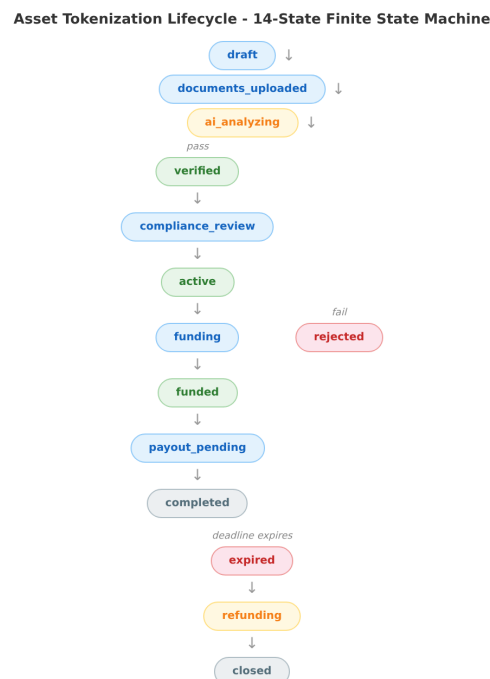


Figure 2. Asset tokenization lifecycle state machine with all 14 states.

5.2.1 Creation Through AI Analysis

Asset creation begins with the owner providing title, description, category (12 predefined options), raise amount (100 to 10,000,000 INR), and listing duration. After validation, the asset enters "draft" status. The owner then uploads 1 to 10 supporting documents (JPG, PNG, WebP, PDF, max 10 MB each) stored with crypto-randomized filenames and SHA-256 hashes for duplicate detection, transitioning to "documents_uploaded". The AI Pipeline then runs five stages: Document Ingestion (classify files, compute quality score), Duplicate Check (SHA-256 comparison), AI Analysis (Gemini 2.0 Flash with fallback engine), Fraud Detection (duplicate docs, unreasonable ratios, low confidence), and Tokenization Recommendation (suggested count and price). Verification requires verified=true, confidence >= 50, and no critical fraud flags.

5.2.2 Tokenization, Investment, and Payout

Upon AI verification, token count is calculated (2-10,000 tokens, risk-adjusted) and token price equals raise amount divided by token count in both INR and AC. A unique escrow account is created. Compliance checks validate raise amount, document count, and description length. An ASSET_CREATE transaction is signed and mined into a block. Investors buy tokens via INVEST transactions moving AC to escrow, with atomic token claims using "UPDATE WHERE owner_id IS NULL" and full rollback on partial claims. When fully funded, the escrow releases 99% to the owner (1% platform fee) via PAYOUT transaction, and AC price receives a 2-5% boost. If the asset expires, all investors are refunded via REFUND transactions and token ownership is cleared.

5.3 Payment and Coin Minting

The payment service integrates four gateways: Razorpay (INR, 1 to 50M INR, HMAC-SHA256 verified), Stripe (INR/USD/EUR/GBP, up to 100M INR, PaymentIntent verified), UPI (INR, up to 200K INR, admin confirmed), and Wire (multi-currency, 100K to 10B INR, admin confirmed). Double-mint prevention uses atomic "UPDATE WHERE status IN (created, pending)" locking. The price recalculation formula after minting: $\text{newPrice} = \text{INITIAL_PRICE} \times (1 + \text{totalSupply}/10000) \times (1 + \text{totalAssetsFunded} \times 0.04)$.

5.4 Trading Engine

The trading engine implements a full order-book exchange with price-time priority matching. Orders are matched when buy price $\geq$ sell price, with the maker (sell) price winning. Self-trading is prevented, partial fills are supported, and market orders use the best available price. A circuit breaker triggers when price moves $>10\%$ within 1 hour, halting all trading until the window resets. Each trade generates a TRADE blockchain transaction. Maximum 50 open orders per user.

5.5 KYC and AML Compliance

The four-tier KYC system controls transaction limits: Tier 0 (Unverified, zero limits), Tier 1 (Basic KYC: 1 doc, 3 trades, 10K INR volume, 7 days; daily 100K/monthly 1M INR), Tier 2 (Full KYC: 10 trades, 100K volume, 30 days; monthly 10M INR), Tier 3 (Institutional: 50 trades, 1M volume, 90 days; monthly 100M INR, annual 1B INR). AML screening checks 7 flag types (HIGH_VALUE, ROUND_AMOUNT, RAPID_SEQUENTIAL, NEW_ACCOUNT_HIGH_VALUE, HIGH_FREQUENCY, DUPLICATE_PATTERN, STRUCTURING), blocking transactions with 3+ simultaneous flags.

5.6 Withdrawal and Settlement

Users with KYC Tier 1+ can request AC-to-fiat withdrawals. The system verifies blockchain balance, deducts 0.5 AC fee, calculates net fiat amount, and creates a pending request. Admin processes by signing TRANSFER and FEE blockchain transactions, mining them, updating DB balances, and decrementing supply. Periodic reconciliation compares internal records with gateway records.

5.7 Admin Operations

The admin panel provides dashboard stats, account freeze/unfreeze, system configuration (fees, circuit breaker), KYC review/approval/rejection, and withdrawal queue management (process, complete, fail). All admin actions are recorded in the hash-chained audit log.

6. Middleware Stack

Every request passes through: express.json (5MB limit) -> sanitizeBody -> generalLimiter -> auditMiddleware -> route handler. Authentication provides three variants: authenticate (requires JWT), optionalAuth (attaches user if token present), and requireRole/requireAdmin for role-based access.

Limiter	Window	Max Requests	Applied To
generalLimiter	60s	100	All routes
authLimiter	60s	10	Login, register
financialLimiter	1s	5	Buy, sell, trade, withdraw
uploadLimiter	60s	10	Document uploads

Table 5. Four-tier token bucket rate limiting configuration.

6.2 Audit Log and Input Validation

Every state-changing request is recorded in a hash-chained audit log where `entry_hash = SHA-256(prev_hash + user_id + action + resource_type + resource_id + timestamp)`. Tampering with any entry breaks the chain, detectable via `verifyAuditChain()`. Input validation and sanitization applies to all endpoints: email format, password strength, amount bounds, and string sanitization stripping script tags, dollar signs, and curly braces.

7. Real-Time Layer

The real-time layer provides instant notifications through an in-memory pub/sub event bus with 14 event types combined with a WebSocket server supporting six channels (price, trades, blocks, assets, orders, users). Clients subscribe/unsubscribe to channels, authenticate via JWT, and receive broadcast events.

Event	Emitted When
BLOCK_MINED	New block mined
TRADE_EXECUTED	Order matched and executed
ORDER_PLACED/CANCELLED	Order book changes
PRICE_UPDATED	AC price changed
ASSET_CREATED/STATUS_CHANGED	Asset lifecycle events
ASSET_FUNDED/TOKEN_PURCHASED	Investment events
COINS_MINTED	AC minted from fiat
CIRCUIT_BREAKER	Trading halted/resumed
ESCROW_LOCKED/RELEASED	Escrow state change
USER_REGISTERED/NOTIFICATION	User events

Table 6. Event types in the in-memory pub/sub event bus.

8. Price Engine

The AC price is driven by three mechanisms. First, supply-driven recalculation on each purchase: $\text{price} = 1.0 \times (1 + \text{totalSupply}/10000) \times (1 + \text{assetsFunded} \times 0.04)$. Second, trade-driven pricing where each matched trade sets the last trade price. Third, natural fluctuation every 15 seconds: $\text{swing} = \text{random}(-0.25\%, +0.25\%)$, $\text{newPrice} = \max(0.001, \text{currentPrice} \times (1 + \text{swing}))$. When an asset is fully funded, an additional boost applies: $\text{boost} = \min(5\%, 2\% + \text{raiseAmount}/1\text{M} \times 3\%)$. All changes are recorded in `price_history` for charting.

9. Fee Structure

The platform generates revenue through five fee types. All fees are recorded in the `fee_ledger` table and aggregated in `economy.total_fees_collected`.

Fee Type	Rate	Collected By
Trading fee	0.1% per trade (both sides)	__PLATFORM_FEE__ wallet
Asset listing fee	1.0 AC	Configured (not enforced)
Capital raise fee	1.0% of raised amount	Deducted from escrow payout
Withdrawal fee	0.5 AC	__PLATFORM_FEE__ wallet
Gateway fees	Varies by gateway	Deducted from fiat amount

Table 7. Complete fee structure.

10. Security Model

The security model implements defense-in-depth across fifteen aspects. Password storage uses PBKDF2-SHA512 with 100K iterations. JWT tokens use HMAC-SHA256 with 15-minute expiry. Blockchain transactions use ECDSA secp256k1. The audit trail uses SHA-256 hash chaining. Four-tier rate limiting protects against abuse. Input sanitization prevents injection. Account lockout (5 failed = 15 min lock) prevents credential stuffing. Double-mint prevention uses atomic DB locks. Race condition protection uses atomic SQL updates with rollback. Webhook verification validates HMAC signatures. The circuit breaker prevents manipulation. KYC/AML enforces limits with 8 flag types. Helmet.js provides HTTP security headers.

Aspect	Implementation
Password storage	PBKDF2-SHA512, 100K iterations, 32-byte salt
JWT tokens	HMAC-SHA256, zero-dependency, 15min access tokens
Blockchain signing	ECDSA secp256k1 (Bitcoin-grade)
Audit trail	SHA-256 hash-chained, tamper-detectable
Rate limiting	4-tier token bucket (100/10/5/10 per window)
Account lockout	5 failed logins, 15-minute lockout
Double-mint prevention	Atomic DB status lock before minting
Race condition protection	UPDATE WHERE owner_id IS NULL with rollback
Circuit breaker	10% price move in 1 hour halts trading
KYC/AML	4-tier limits, 8 AML flags, blocks at 3+ flags
Security headers	Helmet.js (CSP, HSTS, XSS protection)
Input sanitization	HTML entity stripping, length limits
Webhook verification	HMAC signature (Razorpay/Stripe)
Account freezing	Admin can freeze any account
CORS	Configurable allowed origins

Table 8. Complete security model with fifteen defense layers.

11. Deployment Architecture

The platform supports three deployment modes: Development (Node --watch with auto-restart), Production with PM2 cluster mode (auto-forks N workers equal to CPU count, auto-restarts on crash, graceful SIGTERM propagation), and Docker (single container, 15s health checks, 1GB memory limit, 256MB reserved, persistent volumes for data/ and uploads/). The cluster mode uses the Node.js cluster

module where the primary process forks workers and exits.

12. API Route Map

The platform exposes over 40 REST API endpoints. The following table documents the complete API surface with authentication requirements: Auth (JWT required), Fin (financial rate limiter), Admin (admin role), File (file upload), Opt (optional auth).

Method	Path	Auth	Description
GET	/api/config	No	Platform config and stats
GET	/api/dashboard	No	Dashboard stats and activity
POST	/api/auth/register	No	Register with wallet creation
POST	/api/auth/login	No	Login with JWT tokens
POST	/api/auth/refresh	No	Refresh access token
GET	/api/account	Auth	Current user profile
GET	/api/notifications	Auth	User notifications
POST	/api/payment/create-order	Auth+Fin	Create fiat to AC order
POST	/api/payment/verify	Auth	Verify and mint coins
POST	/api/payment/refund	Admin	Reverse completed order
POST	/api/kyc/submit	Auth	Submit KYC document
POST	/api/kyc/verify/:id	Admin	Approve or reject KYC
POST	/api/withdraw/request	Auth+Fin	Request AC to fiat withdrawal
POST	/api/withdraw/process/:id	Admin	Execute on blockchain
POST	/api/withdraw/complete/:id	Admin	Mark completed
GET	/api/assets	Opt	List all assets
POST	/api/assets/create	Auth	Create new asset
POST	/api/assets/:id/documents	Auth	Upload documents
POST	/api/assets/:id/analyze	Auth	Trigger AI analysis
POST	/api/assets/:id/tokens/buy	Auth+Fin	Buy asset tokens
GET	/api/market/orderbook	No	Order book and trades
POST	/api/market/order	Auth+Fin	Place buy/sell order
DELETE	/api/market/order/:id	Auth	Cancel order
GET	/api/portfolio	Auth	Full user portfolio
GET	/api/blockchain/info	No	Chain statistics

Method	Path	Auth	Description
GET	/api/blockchain/validate	No	Chain validity check
GET	/api/admin/stats	Admin	Full admin dashboard
POST	/api/admin/freeze/:userId	Admin	Freeze account
POST	/api/admin/config	Admin	Update system config
GET	/api/economy	No	Economy metrics
GET	/api/economy/price-history	No	Price time series
GET	/health	No	Health check

Table 9. Complete API route map with 33 endpoints.

13. Initialization Sequence

The platform follows a precise 12-step initialization sequence. Step 1 loads .env via dotenv. Step 2 checks cluster mode (production primary forks workers and exits). Step 3 sets up global error boundaries (uncaughtException, unhandledRejection). Step 4 initializes Express with the middleware stack. Step 5 initializes SQLite WASM: creates 25+ tables (IF NOT EXISTS), seeds economy defaults and system_config, and starts the 3-second auto-persist timer.

Step 6 initializes the blockchain by loading chain.json or creating a genesis block. Step 7 initializes the wallet manager by loading wallets.json and ensuring __SYSTEM__ and __PLATFORM_FEE__ wallets exist. Step 8 initializes all eleven service modules (escrow, asset, trading, fee, price, compliance, KYC, payment, settlement, AI pipeline, document processor). Step 9 starts background timers: price fluctuation every 15 seconds, asset deadline check every 60 seconds, session cleanup every hour. Step 10 creates the HTTP and WebSocket servers on PORT (default 4200). Step 12 registers graceful shutdown handlers for SIGTERM/SIGINT that broadcast shutdown via WebSocket, clear timers, persist the database, close the HTTP server, and force exit after a 10-second timeout.